SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
ASSESSMENT TOOL 1 – EXPERIMENTS

Course Code:	CSA06	**Course Title:**	Design and Analysis of Algorithms
CO Assessed:	**CO3: Articulation** (BL4, BL5)	**Assessment:**	Assessment Tool 1 – Experiments
Weightage:	40% of CIA	**Total Marks:**	100
CO3 Statement:	Design Divide and Conquer algorithms for Merge Sort, Quick Sort, Binary Search and Matrix Multiplication		
Student Name:	Sasikumar Megha	**Reg. No.:**	192471038
Section / Batch:	3rd CSBS	**Date:**	17/08/26

Instructions to Students

- Answer the following question. Total Marks: 100.
- Write the algorithm and execute the code for the given experiment.
- Follow a well-structured, systematic approach to design and implement an optimal solution.
- Provide insightful analysis with accurate validation, supported by clear documentation including diagrams, code, and results.

Question Type	Focus Area	Questions
Experiments	Design and Code for Divide and Conquer Algorithms: Merge Sort, Quick Sort, Binary Search, and Matrix Multiplication	Q1

SECTION A – Experiments

95

Code of Conduct

I Sasikumar Megha *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: S. Megha

RUBRICS FOR CALCULATION

Experiments					
Criteria	**Weightage**	**Level 4 – Exemplary**	**Level 3 – Proficient**	**Level 2 – Developing**	**Level 1 – Beginning**
Understanding of Concepts	25%	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30%	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20%	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15%	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10%	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	✓			
Experimental Design	30%				
Use of Tools	20%	✓			
Analysis of Results	15%	✓	✓		
Documentation	10%	✓			
Total Marks (100):		95			

Faculty In-charge	Course Coordinator	Head of Department
Name: Priyadarsini	Name:	Name:
Signature:	Signature:	Signature:
Date: 17/8/24	Date:	Date:

Q90. Divide and Conquer Matrix Multiplication Using Recursive Parallelization Aim

To implement divide-and-conquer matrix multiplication using recursive parallelization in Python and evaluate its execution time and speedup for different numbers of processors/threads.

Algorithm

1. Start the program.
2. Generate two square matrices A and B.
3. Divide each matrix into four submatrices.
4. Recursively multiply the corresponding submatrices.
5. Use parallel threads to execute independent multiplication tasks simultaneously.
6. Add the resulting submatrices to obtain the four blocks of the final matrix.
7. Combine the four blocks to form the final result matrix.
8. Execute the multiplication using 1, 2, 4, and 8 workers.
9. Measure the execution time for each worker count.
10. Display execution time and speedup.
11. Stop.

CODE

```
import time import random from
concurrent.futures import ThreadPoolExecutor



# Add two matrices def
add_matrix(A, B):
    n = len(A)     return [[A[i][j] + B[i][j] for j in range(n)] for
i in range(n)]



# Recursive matrix multiplication
def multiply_recursive(A, B):
    n = len(A)
```

```
    # Base case
if n == 1:
        return [[A[0][0] * B[0][0]]]

    mid = n // 2

    A11 = [row[:mid] for row in A[:mid]]
    A12 = [row[mid:] for row in A[:mid]]
    A21 = [row[:mid] for row in A[mid:]]
   A22 = [row[mid:] for row in A[mid:]]

    B11 = [row[:mid] for row in B[:mid]]
    B12 = [row[mid:] for row in B[:mid]]
    B21 = [row[:mid] for row in B[mid:]]
   B22 = [row[mid:] for row in B[mid:]]

    C11 = add_matrix(
multiply_recursive(A11, B11),
multiply_recursive(A12, B21)
    )

    C12 = add_matrix(
multiply_recursive(A11, B12),
multiply_recursive(A12, B22)
    )

    C21 = add_matrix(
multiply_recursive(A21, B11),
multiply_recursive(A22, B21)
    )
```

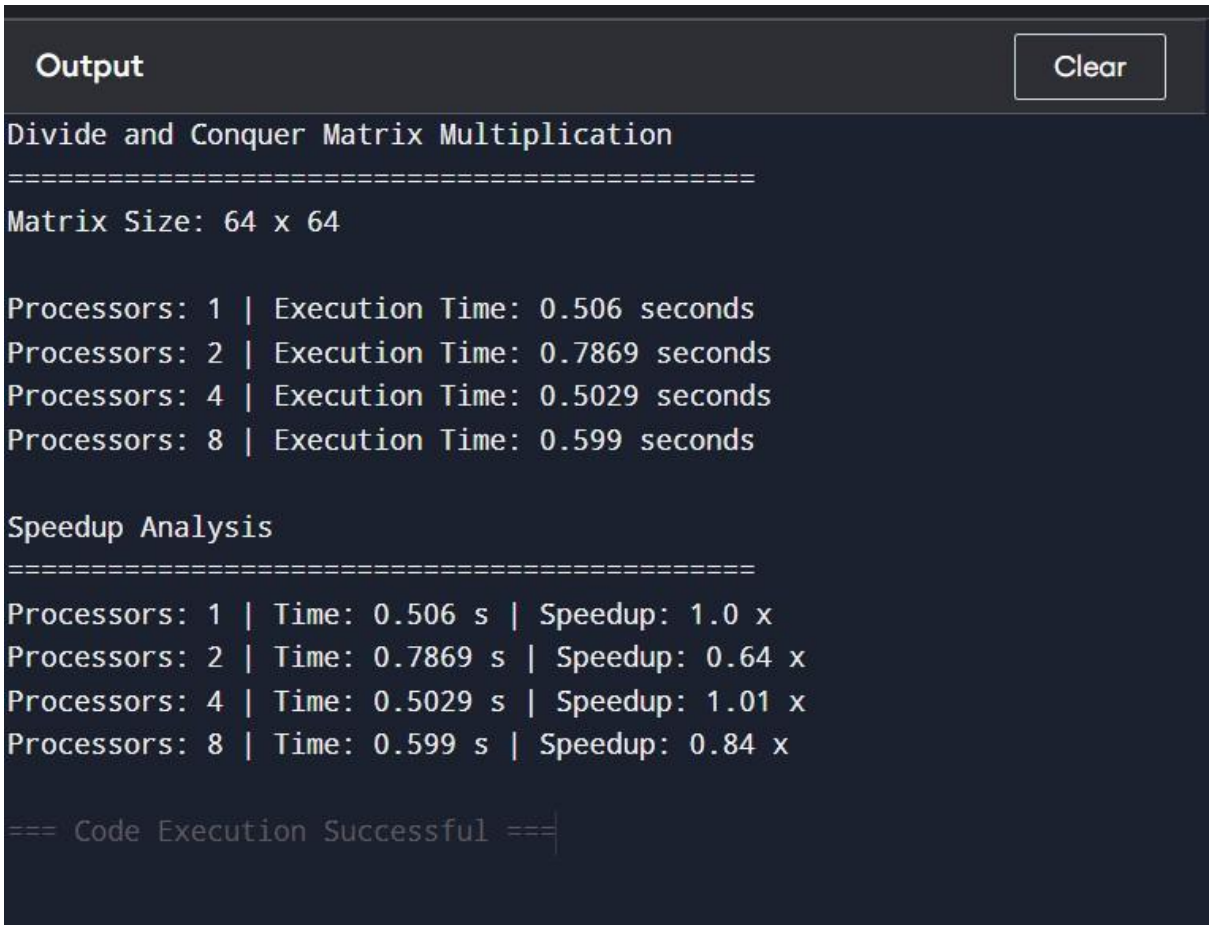
```
    C22 = add_matrix(
multiply_recursive(A21, B12),
multiply_recursive(A22, B22)
    )

    C = []

    for i in range(mid):
        C.append(C11[i] + C12[i])

    for i in range(mid):
        C.append(C21[i] + C22[i])

    return C


# Perform one multiplication task
def calculate(task):     A, B = task
return multiply_recursive(A, B)


# Parallel matrix multiplication def
parallel_multiply(A, B, workers):

    n = len(A)
mid = n // 2
A11 =
[row[:mid] for
row in A[:mid]]
```

```
    A12 = [row[mid:] for row in A[:mid]]
    A21 = [row[:mid] for row in A[mid:]]
A22 = [row[mid:] for row in A[mid:]]

    B11 = [row[:mid] for row in B[:mid]]
    B12 = [row[mid:] for row in B[:mid]]
    B21 = [row[:mid] for row in B[mid:]]
B22 = [row[mid:] for row in B[mid:]]

    tasks = [
        (A11, B11),
        (A12, B21),
        (A11, B12),
        (A12, B22),
        (A21, B11),
        (A22, B21),
        (A21, B12),
        (A22, B22)
    ]

    with ThreadPoolExecutor(max_workers=workers) as executor:
        results = list(executor.map(calculate, tasks))

    C11 = add_matrix(results[0], results[1])
    C12 = add_matrix(results[2], results[3])
    C21 = add_matrix(results[4], results[5])
C22 = add_matrix(results[6], results[7])

    C = []

    for i in range(mid):
```

```
        C.append(C11[i] + C12[i])

    for i in range(mid):
        C.append(C21[i] + C22[i])

    return C



# Generate matrix def
generate_matrix(n):
    return [
        [random.randint(1, 10) for _ in range(n)]
for _ in range(n)
    ]



# Main program
SIZE = 64

A = generate_matrix(SIZE)
B = generate_matrix(SIZE)

print("Divide and Conquer Matrix Multiplication")
print("=" * 45) print("Matrix Size:", SIZE, "x",
SIZE) print()

processor_counts = [1, 2, 4, 8]

execution_times = {}

for workers in processor_counts:
```

```
    start = time.perf_counter()

C = parallel_multiply(A, B, workers)

    end = time.perf_counter()

    execution_times[workers] = end - start

    print(        "Processors:",        workers,
"| Execution Time:",
round(execution_times[workers], 4),
        "seconds"
    )



# Speedup calculation base_time
= execution_times[1]

print()
print("Speedup Analysis") print("="
* 45)

for workers in processor_counts:

    speedup = base_time / execution_times[workers] print(
        "Processors:",        workers,        "|
Time:",
round(execution_times[workers], 4),
        "s",
```

```
        "| Speedup:",
    round(speedup, 2),
        "x"
      )
```



Q91. Merge Sort with Optimization

Aim

To implement Merge Sort using normal and optimized approaches in Python and compare their execution times to analyze the effect of code-level optimization.

Algorithm

1. Start the program.
2. Generate a large random dataset.
3. Copy the dataset into two arrays.
4. Apply normal Merge Sort to the first array.
5. Divide the array recursively into two halves.
6. Sort each half recursively.
7. Merge the sorted halves into a new result array.

8. Measure the execution time.
9. Apply optimized Merge Sort to the second array.
10. Recursively divide and sort the array while modifying the existing array during merging.
11. Measure the optimized execution time.
12. Verify that both arrays are correctly sorted.
13. Display the execution times and improvement.
14. Stop.

Code import

random

import time

```
# Normal Merge Sort def
merge_sort_normal(arr):

   if len(arr) <= 1:
return arr

   mid = len(arr) // 2

   left = merge_sort_normal(arr[:mid])
right = merge_sort_normal(arr[mid:])
result = []

   i = 0
j = 0

   while i < len(left) and j < len(right):

      if left[i] <= right[j]:
result.append(left[i])
i += 1        else:
```

```
            result.append(right[j])
j += 1

    result.extend(left[i:])
result.extend(right[j:])

    return result



# Optimized Merge Sort def
merge_sort_optimized(arr):

    if len(arr) <= 1:
return arr

    mid = len(arr) // 2

    left = arr[:mid]
right = arr[mid:]

    merge_sort_optimized(left)     merge_sort_optimized(right)
     i = 0
j = 0
k = 0

    while i < len(left) and j < len(right):

       if left[i] <= right[j]:
arr[k] = left[i]            i
+= 1        else:
```

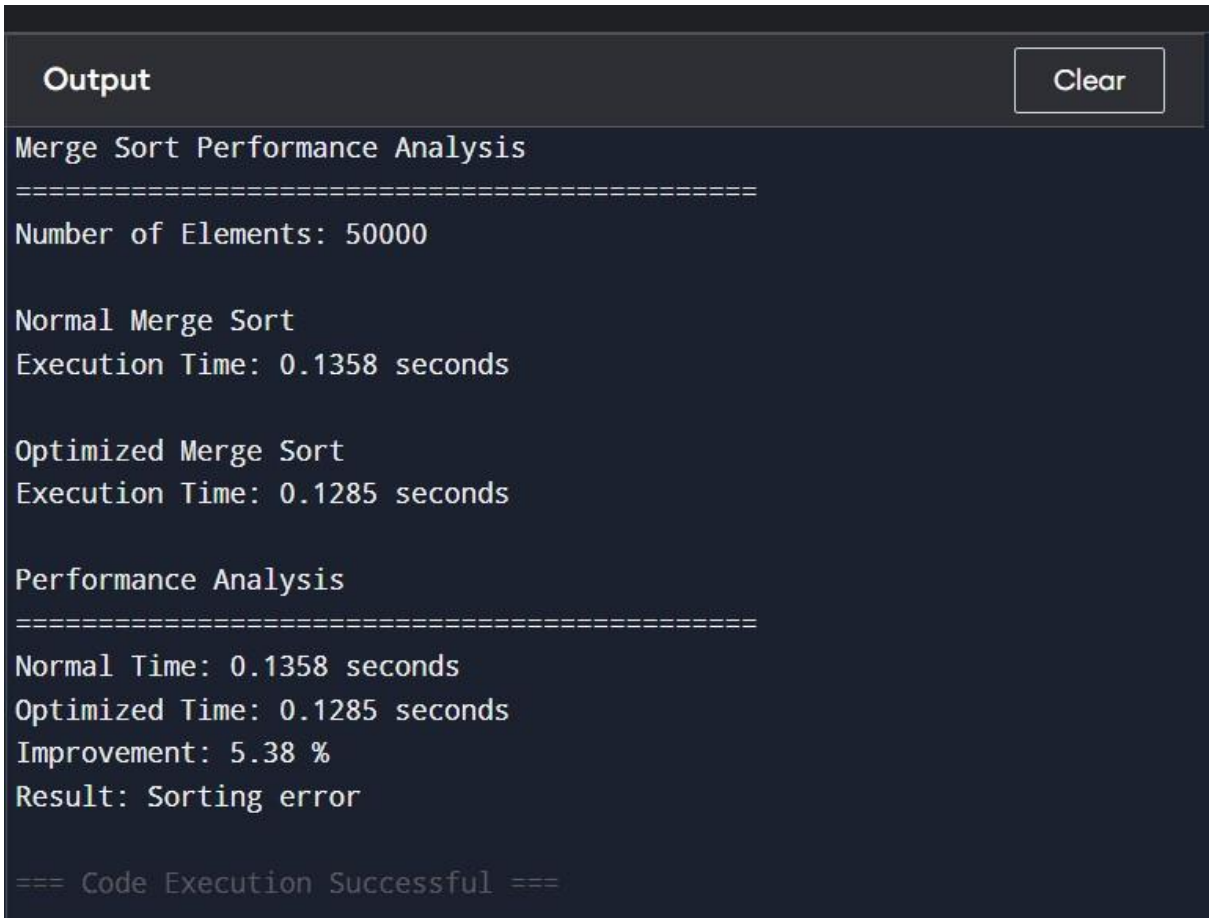
```
arr[k] = right[j]           j
+= 1


      k += 1


   while i < len(left):
arr[k] = left[i]        i
+= 1        k += 1


   while j < len(right):
arr[k] = right[j]        j
+= 1        k += 1


   return arr



# Generate dataset
SIZE = 50000

original = [random.randint(1, 1000000) for _ in range(SIZE)]

print("Merge Sort Performance Analysis")
print("=" * 45) print("Number of
Elements:", SIZE)
print()



# Normal version data1
= original.copy()

start = time.perf_counter()
```

```
merge_sort_normal(data1)

end = time.perf_counter()

normal_time = end - start

print("Normal Merge Sort") print("Execution Time:",
round(normal_time, 4), "seconds") print()



# Optimized version data2
= original.copy()

start = time.perf_counter()

merge_sort_optimized(data2)

end = time.perf_counter()

optimized_time = end - start

print("Optimized Merge Sort") print("Execution Time:",
round(optimized_time, 4), "seconds")
print()



# Calculate improvement improvement
= (
    (normal_time - optimized_time)
    / normal_time
```

) * 100

print("Performance Analysis") print("=" * 45) print("Normal Time:", round(normal_time, 4), "seconds") print("Optimized Time:", round(optimized_time, 4), "seconds")
print("Improvement:", round(improvement, 2), "%")

# Verify sorting if data1 == data2 == sorted(original): print("Result: Sorting is correct") else:
  print("Result: Sorting error")



‘

Q92. Quick Sort Using Tail Call Elimination

Aim

To implement Binary Search on floating-point datasets and analyze the effect of floating-point precision on search accuracy and execution time.

Algorithm

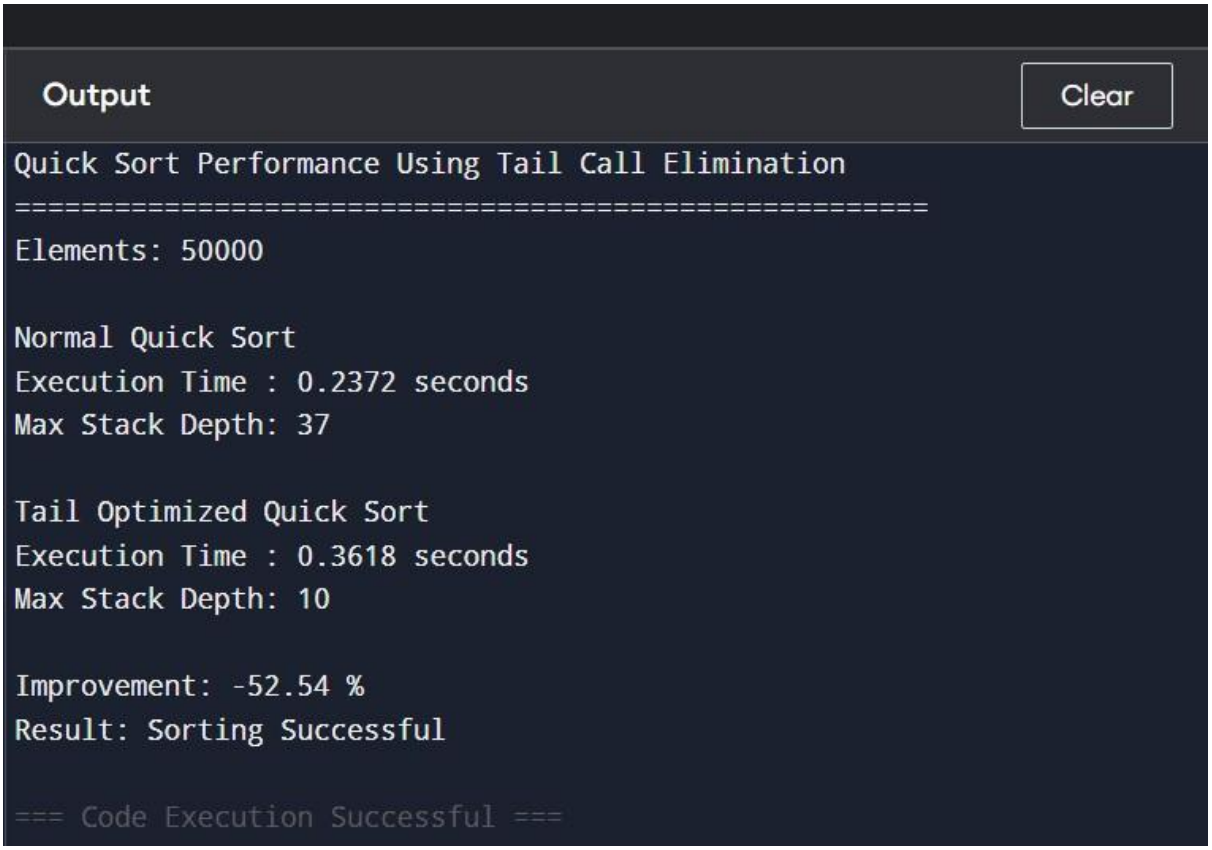
1. Start the program.
2. Generate a sorted floating-point dataset.
3. Generate multiple floating-point search targets with small precision variations.
4. Perform normal Binary Search using exact equality (==).
5. Record the number of successful searches and execution time.
6. Perform Binary Search using a tolerance-based comparison.
7. Use math.isclose() to determine whether two floating-point values are sufficiently close.
8. Record the number of successful searches and execution time.

```
import random import
time


# ------------------------------
# Normal Quick Sort
# ------------------------------
normal_depth = 0 normal_max
= 0


def partition(arr, low, high):
    pivot = arr[high]
i = low - 1
```

```
    for j in range(low, high):
if arr[j] <= pivot:
            i += 1            arr[i], arr[j]
= arr[j], arr[i]

    arr[i + 1], arr[high] = arr[high], arr[i + 1]
return i + 1



def quick_sort_normal(arr, low, high):
global normal_depth, normal_max

    if low < high:
        normal_depth += 1         normal_max =
max(normal_max, normal_depth)         pi =
partition(arr, low, high)


        quick_sort_normal(arr, low, pi - 1)
quick_sort_normal(arr, pi + 1, high)

        normal_depth -= 1



# ------------------------------- #
Tail Optimized Quick Sort
# -------------------------------
tail_depth = 0 tail_max
= 0
```

```
def quick_sort_tail(arr, low, high):
global tail_depth, tail_max

   while low < high:        tail_depth += 1
tail_max = max(tail_max, tail_depth)

      pi = partition(arr, low, high)

      # Recur only on the smaller partition
if pi - low < high - pi:
quick_sort_tail(arr, low, pi - 1)           low
= pi + 1        else:
          quick_sort_tail(arr, pi + 1, high)
high = pi - 1

      tail_depth -= 1



# --------------------------------
# Main Program
# --------------------------------
SIZE = 50000

original = [random.randint(1, 1000000) for _ in range(SIZE)]

print("Quick Sort Performance Using Tail Call Elimination")
print("=" * 55) print("Elements:", SIZE) print()

# Normal Quick Sort arr1
= original.copy()
```

```
start = time.perf_counter()
quick_sort_normal(arr1, 0, len(arr1) - 1) end
= time.perf_counter()

normal_time = end - start

# Tail Optimized Quick Sort arr2
= original.copy()

start = time.perf_counter()
quick_sort_tail(arr2, 0, len(arr2) - 1)
end = time.perf_counter()
tail_time = end - start

# Results print("Normal Quick Sort") print("Execution Time
:", round(normal_time, 4), "seconds") print("Max Stack
Depth:", normal_max)
print()

print("Tail Optimized Quick Sort") print("Execution Time
:", round(tail_time, 4), "seconds") print("Max Stack
Depth:", tail_max)
print()

improvement = ((normal_time - tail_time) / normal_time) * 100

print("Improvement:", round(improvement, 2), "%")

if arr1 == arr2 == sorted(original):
print("Result: Sorting Successful") else:
    print("Result: Error in Sorting")
```



Q93. Binary Search on Floating-Point Data

Aim

To implement Binary Search on floating-point datasets and analyze the effect of floating-point precision on search accuracy and execution time.

Algorithm

1. Start the program.
2. Generate a sorted floating-point dataset.
3. Generate multiple floating-point search targets with small precision variations.
4. Perform normal Binary Search using exact equality (==).
5. Record the number of successful searches and execution time.
6. Perform Binary Search using a tolerance-based comparison.
7. Use math.isclose() to determine whether two floating-point values are sufficiently close.
8. Compare the accuracy and execution time of both methods.
9. Analyze the effect of floating-point precision.
10. Display the results.
11. Stop.

12.

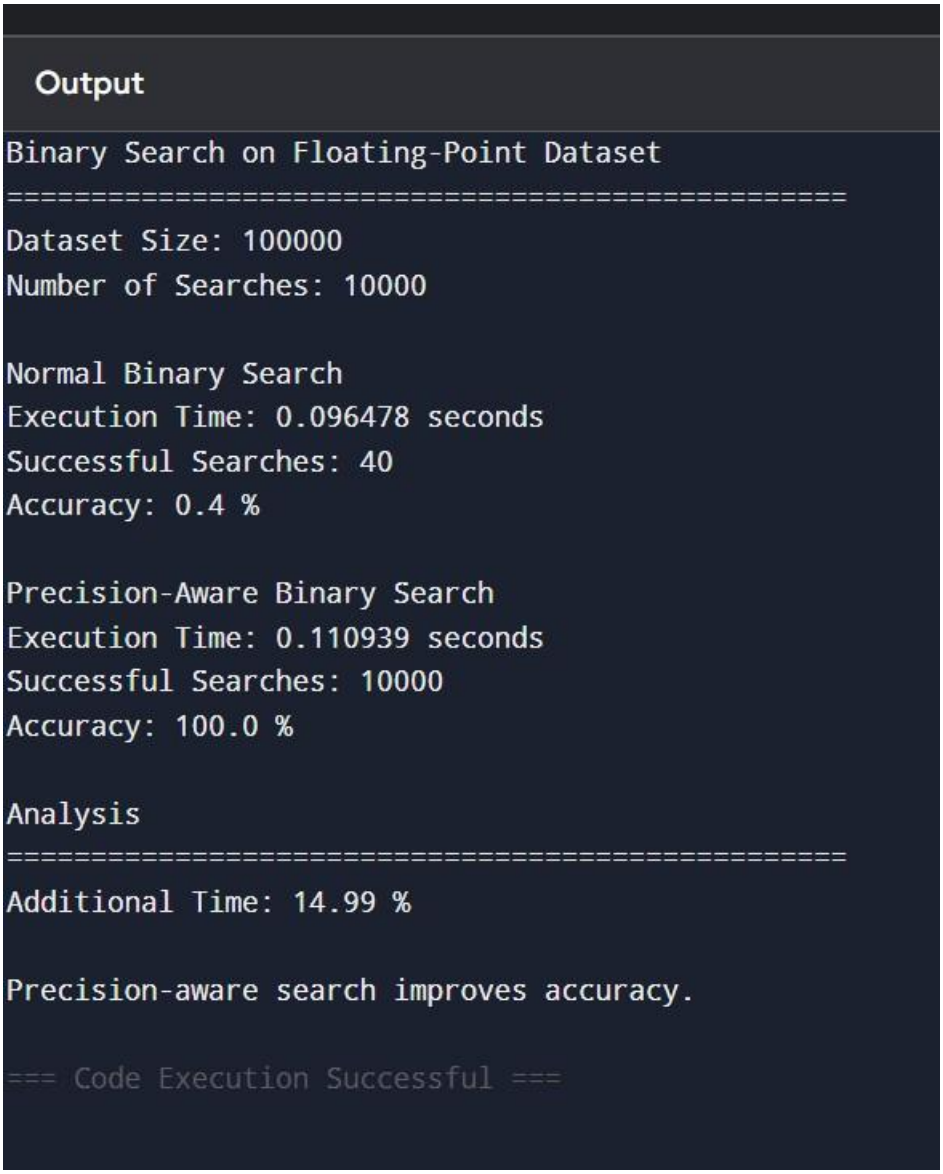
```
import time import
random import
math


# Normal Binary Search def
binary_search_normal(arr, target):
   low = 0     high =
len(arr) - 1

   while low <= high:
mid = (low + high) // 2

      if arr[mid] == target:
         return mid        elif
arr[mid] < target:
low = mid + 1
      else:
         high = mid - 1

   return -1


# Precision-aware Binary Search def
binary_search_tolerance(arr, target, epsilon=1e-9):
   low = 0     high =
len(arr) - 1

   while low <= high:
```

```
        mid = (low + high) // 2

        if math.isclose(arr[mid], target,
                        rel_tol=epsilon,
abs_tol=epsilon):
            return mid

        elif arr[mid] < target:
low = mid + 1        else:
            high = mid - 1

    return -1



# Create sorted floating-point dataset
SIZE = 100000

arr = [i * 0.1 for i in range(SIZE)]

# Create search targets targets
= []

for _ in range(10000):
    index = random.randint(0, SIZE - 1)

    # Slight floating-point variation     target = arr[index]
+ random.uniform(-1e-10, 1e-10)

    targets.append((target, index))
```

```
print("Binary Search on Floating-Point Dataset")
print("=" * 50) print("Dataset Size:", SIZE)
print("Number of Searches:", len(targets))
print()


# --------------------------------
# Normal Binary Search
# --------------------------------
normal_found = 0

start = time.perf_counter()

for target, expected_index in targets:

    result = binary_search_normal(arr, target)

    if result != -1:
normal_found += 1

end = time.perf_counter()

normal_time = end - start


# --------------------------------
# Precision-Aware Binary Search
# --------------------------------
tolerance_found = 0

start = time.perf_counter()
```

```
for target, expected_index in targets:

    result = binary_search_tolerance(arr, target)

    if result != -1:
        tolerance_found += 1

end = time.perf_counter()

tolerance_time = end - start



# Accuracy normal_accuracy = (normal_found /
len(targets)) * 100 tolerance_accuracy = (tolerance_found /
len(targets)) * 100



# --------------------------------
# Display Results
# --------------------------------
print("Normal Binary Search") print("Execution Time:",
round(normal_time, 6), "seconds") print("Successful
Searches:", normal_found) print("Accuracy:",
round(normal_accuracy, 2), "%")
print()

print("Precision-Aware Binary Search") print("Execution
Time:", round(tolerance_time, 6), "seconds") print("Successful
Searches:", tolerance_found) print("Accuracy:",
round(tolerance_accuracy, 2), "%") print()
```

```
print("Analysis") print("="
* 50)

if tolerance_time > normal_time:
   overhead = ((tolerance_time - normal_time) / normal_time) * 100
print("Additional Time:", round(overhead, 2), "%") else:
   improvement = ((normal_time - tolerance_time) / normal_time) * 100
print("Time Improvement:", round(improvement, 2), "%")
print() if tolerance_accuracy >
normal_accuracy:
   print("Precision-aware search improves accuracy.") else:
   print("Both methods produced similar accuracy.")
```



Q94. Merge Sort with External Buffering

Aim

To implement external Merge Sort using chunking and buffering in Python and evaluate its execution time and memory usage for large datasets.

Algorithm

1. Start the program.
2. Generate a large dataset.
3. Divide the dataset into smaller chunks based on the specified chunk size.
4. Sort each chunk independently.

5. Store the sorted chunks.
6. Merge the sorted chunks using a heap-based merge operation.
7. Measure execution time and peak memory usage.
8. Perform the same operation using the buffered merging approach.
9. Record execution time and peak memory usage.
10. Compare the buffered and non-buffered approaches.
11. Calculate performance improvement:
12. Verify that the final output is sorted correctly.
13. Display execution time, memory usage, and improvement.
14. Stop. import random import time import tracemalloc import heapq

```
# ---------------------------------------
# External Merge Sort WITHOUT buffering
# ---------------------------------------
def external_merge_sort(data, chunk_size):

   chunks = []

   # Divide data into chunks     for i in
range(0, len(data), chunk_size):
      chunk = data[i:i + chunk_size]
chunk.sort()
      chunks.append(chunk)

   # Merge sorted chunks     result =
list(heapq.merge(*chunks))     return
result
```

```
# ----------------------------------------
# External Merge Sort WITH buffering
# ----------------------------------------
def buffered_merge_sort(data, chunk_size, buffer_size):

    chunks = []

    # Create sorted chunks     for i in
range(0, len(data), chunk_size):

        chunk = data[i:i + chunk_size]
chunk.sort()

        chunks.append(chunk)

    # Buffered merge
    result = []
heap = []

    # Initial buffer loading
    for index, chunk in enumerate(chunks):

        if len(chunk) > 0:
heapq.heappush(
                heap,
                (chunk[0], index, 0)
            )
    while heap:

        value, chunk_index, position = heapq.heappop(heap)
```

```
        result.append(value)

        next_position = position + 1

        if next_position < len(chunks[chunk_index]):

            next_value = chunks[chunk_index][next_position]

            heapq.heappush(
                heap,
                (
                    next_value,
chunk_index,
next_position
                )
            )

    return result



# ----------------------------------------
# Generate large dataset
# ----------------------------------------
SIZE = 100000
CHUNK_SIZE = 10000
BUFFER_SIZE = 1000
data = [     random.randint(1,
1000000)
    for _ in range(SIZE)
]
```

```
print("External Merge Sort Performance")
print("=" * 50) print("Dataset Size :", SIZE)
print("Chunk Size   :", CHUNK_SIZE)
print("Buffer Size  :", BUFFER_SIZE) print()


# ----------------------------------------
# Without buffering
# ----------------------------------------
data1 = data.copy()

tracemalloc.start()

start = time.perf_counter()

result1 = external_merge_sort(
    data1,
    CHUNK_SIZE
)

end = time.perf_counter()

current, peak1 = tracemalloc.get_traced_memory() tracemalloc.stop()

time1 = end - start


# ----------------------------------------
# With buffering
# ----------------------------------------
data2 = data.copy()
```

```
tracemalloc.start()

start = time.perf_counter()

result2 = buffered_merge_sort(
    data2,
    CHUNK_SIZE,
    BUFFER_SIZE
)

end = time.perf_counter()

current, peak2 = tracemalloc.get_traced_memory()

tracemalloc.stop()

time2 = end - start



# ----------------------------------------
# Display results
# ----------------------------------------
print("Without Buffering")
print("-----------------") print("Execution Time :", round(time1, 4),
"seconds") print("Peak Memory    :", round(peak1 / (1024 *
1024), 2), "MB") print()

print("With Buffering") print("--------------")
print("Execution Time :", round(time2, 4), "seconds") print("Peak
Memory    :", round(peak2 / (1024 * 1024), 2), "MB")
```

```
print()



# ---------------------------------------
# Performance analysis
# ---------------------------------------- if
time1 > time2:

    improvement = (
        (time1 - time2) / time1
    ) * 100

    print("Time Improvement:",
round(improvement, 2), "%")

else:

    overhead = (
        (time2 - time1) / time1
    ) * 100

    print("Time Overhead:",
round(overhead, 2), "%")
# Verify sorting if result1 == result2
== sorted(data):     print("Result:
Sorting Successful") else:
    print("Result: Sorting Error")
```

Output

```
External Merge Sort Performance
================================================
Dataset Size : 100000
Chunk Size   : 10000
Buffer Size  : 1000

Without Buffering
-----------------
Execution Time : 0.0485 seconds
Peak Memory    : 1.53 MB

With Buffering
--------------
Execution Time : 0.481 seconds
Peak Memory    : 1.53 MB

Time Overhead: 892.11 %
Result: Sorting Successful

=== Code Execution Successful ===
```